# 11.1 Routing Fundamentals

## Introduction

Routing is a core feature of Angular that enables navigation between different views or pages in a Single Page Application (SPA). Unlike traditional multi-page applications where each page requires a full server request and browser reload, Angular routing allows users to navigate between different views without leaving the application or reloading the page. This creates a smooth, fast, and responsive user experience.

## What is Routing?

Routing is the mechanism that maps URL paths to specific components in your Angular application. When a user navigates to a particular URL, the Angular router displays the corresponding component's view. This process happens entirely on the client side without requesting new pages from the server.

### Traditional vs SPA Navigation

#### Traditional Multi-Page Application

- User clicks a link
- Browser sends request to server
- Server processes request and sends complete HTML page
- Browser discards old page and loads new page
- CSS and JavaScript files reload
- Entire page refreshes (slower)

#### Single Page Application with Routing

- User clicks a link
- Angular router intercepts the navigation
- Router loads the appropriate component
- Only the view area changes
- No page reload required (faster)

- Application state is preserved

> ⚠ **Important:**
>
> Routing transforms your Angular application into a true SPA by enabling seamless navigation between different views while maintaining application state and providing a fast, responsive user experience.

## Module-Based vs Standalone Components

Angular offers two approaches for building applications: the traditional **module-based** approach and the modern **standalone components** approach. Understanding both is crucial for working with different Angular projects.

### Module-Based Approach (Traditional)

- Components are declared in NgModules
- RouterModule is configured using `forRoot()` and `forChild()`
- Requires separate routing modules
- More boilerplate code
- Still widely used in existing projects

### Standalone Components Approach (Modern)

- Components are self-contained with `standalone: true`
- No NgModule declarations needed
- Direct imports in component metadata
- Simplified routing configuration
- Recommended for new Angular projects (Angular 14+)
- Easier to understand and maintain
- Better tree-shaking and smaller bundle sizes

⚠ Important:

**Which Approach to Use?**

- **New Projects:** Use standalone components (recommended by Angular team)
- **Existing Projects:** Can be gradually migrated to standalone
- **Learning:** Understand both approaches as you'll encounter both in the industry

## Setting Up Routing

To enable routing in an Angular application, you need to configure the RouterModule. We'll cover both module-based and standalone approaches.

### Approach 1: Module-Based Routing (Traditional)

In the module-based approach, Angular provides two methods: `forRoot()` for the root module and `forChild()` for feature modules.

#### Step 1: Creating a Basic Routing Configuration

First, let's create a simple application with routing. We'll start by creating a few components to navigate between.

```
// home.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  template: `
    <div>
      <h2>Welcome to the Home Page</h2>
      <p>This is the main landing page of our application.</p>
    </div>
  `
})
export class HomeComponent { }
```

```
// about.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-about',
  template: `
    <div>
      <h2>About Us</h2>
      <p>Learn more about our company and mission.</p>
    </div>
  `
})
export class AboutComponent { }
```

```
// contact.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-contact',
  template: `
    <div>
      <h2>Contact Us</h2>
      <p>Email: info@company.com</p>
      <p>Phone: +1-555-0123</p>
    </div>
  `
})
export class ContactComponent { }
```

## Step 2: Defining Routes

Routes are defined as an array of route configuration objects. Each route maps a URL path to a component.

```
// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { AboutComponent } from './about.component';
import { ContactComponent } from './contact.component';
```

```
const routes: Routes = [
  { path: '', component: HomeComponent },            // Default route
  { path: 'home', component: HomeComponent },       // /home URL
  { path: 'about', component: AboutComponent },     // /about URL
  { path: 'contact', component: ContactComponent }  // /contact URL
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

> ⚠ **Important:**
>
> **RouterModule.forRoot()** is used in the root module (AppModule) to configure the router with the application-wide routes. Use **forRoot()** only once in your application.

### Step 3: Registering the Routing Module

```
// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { HomeComponent } from './home.component';
import { AboutComponent } from './about.component';
import { ContactComponent } from './contact.component';

@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    AboutComponent,
    ContactComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule  // Import the routing module
  ],
  providers: [],
```

```
  bootstrap: [AppComponent]
})
export class AppModule { }
```

## Approach 2: Standalone Components Routing (Modern)

With standalone components, routing is simpler and more straightforward. No modules or routing modules are needed.

### Step 1: Creating Standalone Components

```
// home.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  standalone: true,  // ← Makes component standalone
  template: `
    <div>
      <h2>Welcome to the Home Page</h2>
      <p>This is the main landing page of our application.</p>
    </div>
  `
})
export class HomeComponent { }
```

```
// about.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-about',
  standalone: true,  // ← Standalone component
  template: `
    <div>
      <h2>About Us</h2>
      <p>Learn more about our company and mission.</p>
    </div>
  `
})
export class AboutComponent { }
```

```
// contact.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-contact',
  standalone: true,  // ← Standalone component
  template: `
    <div>
      <h2>Contact Us</h2>
      <p>Email: info@company.com</p>
      <p>Phone: +1-555-0123</p>
    </div>
  `
})
export class ContactComponent { }
```

### Step 2: Defining Routes (app.routes.ts)

With standalone components, routes are defined in a simple TypeScript file without NgModule.

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from  - Module-Based
```

```
// app.component.ts (Module-Based)
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>My Angular Application</h1>

      <nav>
        <a routerLink="/">Home</a> |
        <a routerLink="/about">About</a> |
        <a routerLink="/contact">Contact</a>
      </nav>

      <hr>
      <router-outlet></router-outlet>
    </div>
```

```
  `
})
export class AppComponent { }
```

## Basic routerLink Usage - Standalone

```
// app.component.ts (Standalone)
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink],  // ← Must import directives:
HomeComponent },          // Default route
  { path: 'home', component: HomeComponent },       // /home URL
  { path: 'about', component: AboutComponent },     // /about URL
  { path: 'contact', component: ContactComponent }  // /contact URL
];
```

### Step 3: Bootstrap with Standalone Configuration (main.ts)

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes)  // ← Provide routing configuration
  ]
}).catch(err => console.error(err));
```

### Step 4: App Component (Standalone)

```
// app.component.ts
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink],  // ← Import routing directives
  template: `
    <div>
      <h1>My Angular Application</h1>

      <nav>
        <a routerLink="/home">Home</a> |
        <a routerLink="/about">About</a> |
        <a routerLink="/contact">Contact</a>
      </nav>

      <hr>
      <router-outlet></router-outlet>

      <hr>
      <footer>
        <p>© 2024 My Company</p>
      </footer>
    </div>
  `
})
export class AppComponent { }
```

⚠ **Important:**

**Key Differences in Standalone Approach:**

- No `@NgModule` required
- Use `standalone: true` in component decorator
- Import routing directives directly in `imports` array
- Use `provideRouter()` instead of `RouterModule.forRoot()`
- Bootstrap with `bootstrapApplication()` instead of platformBrowserDynamic

### Comparison: Module-Based vs Standalone

| Aspect | Module-Based | Standalone |
|---|---|---|
| **Setup Complexity** | More files (module, routing module) | Fewer files, simpler structure |
| **Component Declaration** | Declared in NgModule | Self-contained with `standalone: true` |
| **Router Configuration** | `RouterModule.forRoot(routes)` | `provideRouter(routes)` |
| **Bootstrap** | `platformBrowserDynamic().bootstrapModule()` | `bootstrapApplication()` |
| **Imports** | Module-level imports | Component-level imports |
| **Code Amount** | More boilerplate | Less boilerplate |
| **Learning Curve** | Higher (understand modules) | Lower (more intuitive) |

## Router Outlet

The `<router-outlet>` directive acts as a placeholder where the router displays the component that matches the current route. Think of it as a dynamic container that changes its content based on the current URL.

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>My Angular Application</h1>

      <nav>
        <a href="/home">Home</a> |
        <a href="/about">About</a> |
        <a href="/contact">Contact</a>
      </nav>
```

```
      <hr>

      <!-- Routed component will be displayed here -->
      <router-outlet></router-outlet>

      <hr>
      <footer>
        <p>© 2024 My Company</p>
      </footer>
    </div>
  `
})
export class AppComponent { }
```

**Output:**

**Browser Display (when navigating to /home):**

# My Angular Application

Home | About | Contact

---

## Welcome to the Home Page

This is the main landing page of our application.

---

© 2024 My Company

**How Router Outlet Works:**

- The content above `<router-outlet>` remains constant (header, navigation)
- The routed component appears where `<router-outlet>` is placed
- The content below `<router-outlet>` also remains constant (footer)

- Only the component inside the outlet changes when navigating

## Navigation with routerLink

While using standard `href` attributes works, it causes the browser to reload the entire page, defeating the purpose of a SPA. Angular provides the `routerLink` directive for navigation without page reloads.

### Basic routerLink Usage

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>My Angular Application</h1>

      <nav>
        <a routerLink="/">Home</a> |
        <a routerLink="/about">About</a> |
        <a routerLink="/contact">Contact</a>
      </nav>

      <hr>
      <router-outlet></router-outlet>
    </div>
  `
})
export class AppComponent { }
```

> ⚠ **Important:**
>
> **routerLink vs href:**
> Use `routerLink` instead of `href` for internal navigation. The `routerLink` directive prevents page reloads and maintains application state, making your application faster and more responsive.

### routerLink with Array Syntax

The `routerLink` directive also accepts an array of path segments. This is useful for constructing routes programmatically.

```
// navigation.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-navigation', - Module-Based
    // app.component.ts (Module-Based)
    <nav>
      <!-- String syntax -->
      <a routerLink="/home">Home</a>

      <!-- Array syntax -->
      <a [routerLink]="['/about']">About</a>
      <a [routerLink]="['/contact']">Contact</a>

      <!-- Dynamic route with variable -->
      <a [routerLink]="['/home']">{{ homeLabel }}</a>
    </nav>
  `
})
export class NavigationComponent {
  homeLabel: string = 'Go to Home';
}
```

> ⚠ **Important:**
>
> When using array syntax with `[routerLink]`, you need property binding (square brackets). The string syntax `routerLink="/path"` does not require brackets.

## routerLinkActive Directive

The `routerLinkActive` directive automatically adds a CSS class to an element when its associated route is active. This is useful for highlighting the current page in a navigation menu.

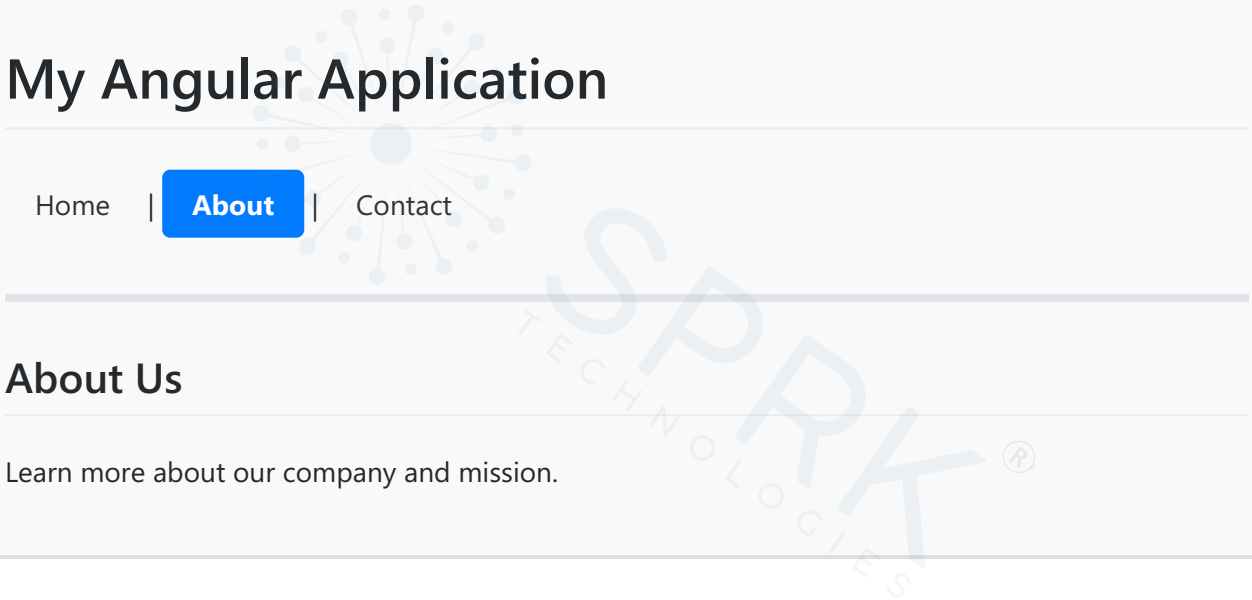
## Basic routerLinkActive

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>My Angular Application</h1>

      <nav>
        <a routerLink="/"
           routerLinkActive="active"
           [routerLinkActiveOptions]="{exact: true}">
          Home
        </a> |
        <a routerLink="/about"
           routerLinkActive="active">
          About
        </a> |
        <a routerLink="/contact"
           routerLinkActive="active">
          Contact
        </a>
      </nav>

      <hr>
      <router-outlet></router-outlet>
    </div>
  `,
  styles: [`
    nav a {
      padding: 10px 15px;
      text-decoration: none;
      color: #333;
      border: 1px solid transparent;
    }

    nav a.active {
      color: #fff;
      background-color: #007bff;
      border-color: #007bff;
      border-radius: 4px;
      font-weight: bold;
    }
```

```
    nav a:hover {
      background-color: #0056b3;
      color: #fff;
      border-radius: 4px;
    }
  `]
})
export class AppComponent { }
```

Output:

Browser Display (when on /about page):




### Understanding routerLinkActiveOptions

The `routerLinkActiveOptions` directive configures when the active class should be applied:

```
// Example demonstrating exact match requirement
<a routerLink="/"
   routerLinkActive="active"
   [routerLinkActiveOptions]="{exact: true}">
  Home
</a>
```

| Option | Value | Behavior |
| --- | --- | --- |
| exact | true | Active class applied only when URL exactly matches the route |
| exact | false | Active class applied when URL starts with the route path (default) |

**Why use exact: true?**

The root path "/" is a prefix of all routes ("/about", "/contact", etc.). Without `exact: true`, the home link would always be highlighted because every route starts with "/". Setting `exact: true` ensures the home link is only active when the URL is exactly "/".

### Multiple Active Classes

```
// Applying multiple CSS classes when route is active
<a routerLink="/about"
   routerLinkActive="active-link highlighted">
  About
</a>
```

## Route Configuration Options

Each route in the routes array can have several configuration properties. Let's explore the most commonly used ones.

### Path and Component

```
const routes: Routes = [
  // Basic route: maps path to component
  { path: 'products', component: ProductsComponent },

  // Empty path (default/home route)
  { path: '', component: HomeComponent },
```

```
  // Path with multiple segments
  { path: 'admin/dashboard', component: AdminDashboardComponent }
];
```

## Redirects

You can redirect from one route to another using the `redirectTo` property.

```
const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },

  // Redirect old URL to new URL
  { path: 'old-about', redirectTo: '/about', pathMatch: 'full' },
  { path: 'about', component: AboutComponent }
];
```

> ⚠ **Important:**
>
> **pathMatch: 'full'** ensures the redirect only happens when the entire URL matches. Without it, the redirect might occur on partial matches, causing unexpected behavior.

## Wildcard Route (404 Page)

The wildcard route `**` matches any URL that doesn't match previous route definitions. Use it to create a 404 "page not found" component.

```
// page-not-found.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-page-not-found',
  template: `
    <div style="text-align: center; padding: 50px;">
      <h1>404 - Page Not Found</h1>
      <p>The page you are looking for does not exist.</p>
      <a routerLink="/">Go to Home</a>
    </div>
```

```
  `
})
export class PageNotFoundComponent { }
```

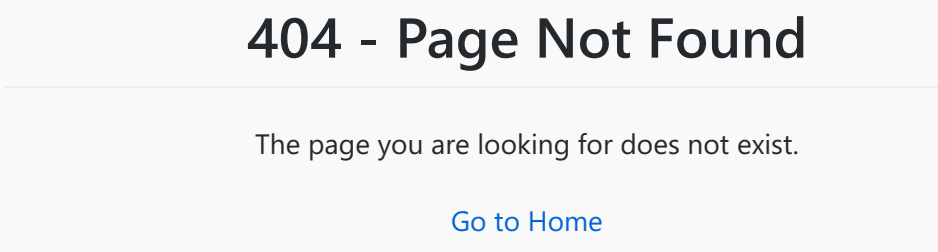
```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'contact', component: ContactComponent },

  // Wildcard route - MUST be last
  { path: '**', component: PageNotFoundComponent }
];
```

⚠ Important:

**Important:** The wildcard route must be placed last in the routes array because routes are matched in the order they are defined. If you place it earlier, it will match all routes and prevent other routes from working.

Output:

**Browser Display (when navigating to /unknown-page):**



## Child Routes

Child routes allow you to create nested views within a parent component. This is useful for creating layouts with multiple levels of navigation. Both approaches support child routes, but the syntax differs slightly.

### Child Routes - Module-Based

```
// products.component.ts (Module-Based)
import { Component } from '@angular/core';

@Component({
  selector: 'app-products',
  template: `
    <div>
      <h2>Products Section</h2>

      <nav>
        <a routerLink="electronics">Electronics</a> |
        <a routerLink="clothing">Clothing</a> |
        <a routerLink="books">Books</a>
      </nav>

      <hr>

      <!-- Child route components appear here -->
      <router-outlet></router-outlet>
    </div>
  `
})
export class ProductsComponent { }
```

```
// electronics.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-electronics',
  template: `
    <div>
      <h3>Electronics</h3>
      <ul>
        <li>Laptops</li>
```

```
        <li>Smartphones</li>
        <li>Tablets</li>
      </ul>
    </div>
  `
})
export class ElectronicsComponent { }
```

```
// clothing.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-clothing',
  template: `
    <div>
      <h3>Clothing</h3>
      <ul>
        <li>Shirts</li>
        <li>Pants</li>
        <li>Shoes</li>
      </ul>
    </div>
  `
})
export class ClothingComponent { }
```

```
// books.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-books',
  template: `
    <div>
      <h3>Books</h3>
      <ul>
        <li>Fiction</li>
        <li>Non-Fiction</li>
        <li>Science</li>
      </ul>
    </div>
```

```
  `
})
export class BooksComponent { }
```

## Configuring Child Routes (Module-Based)

```
// app-routing.module.ts (Module-Based)
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'products',
    component: ProductsComponent,
    children: [
      { path: 'electronics', component: ElectronicsComponent },
      { path: 'clothing', component: ClothingComponent },
      { path: 'books', component: BooksComponent }
    ]
  },
  { path: '**', component: PageNotFoundComponent }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

## Child Routes - Standalone

```
// products.component.ts (Standalone)
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-products',
  standalone: true,
  imports: [RouterOutlet, RouterLink],  // ← Import routing directives
  template: `
    <div>
```

```
      <h2>Products Section</h2>

      <nav>
        <a routerLink="electronics">Electronics</a> |
        <a routerLink="clothing">Clothing</a> |
        <a routerLink="books">Books</a>
      </nav>

      <hr>
      <router-outlet></router-outlet>
    </div>
  `
})
export class ProductsComponent { }
```

```
// electronics.component.ts (Standalone)
import { Component } from '@angular/core';

@Component({
  selector: 'app-electronics',
  standalone: true,
  template: `
    <div>
      <h3>Electronics</h3>
      <ul>
        <li>Laptops</li>
        <li>Smartphones</li>
        <li>Tablets</li>
      </ul>
    </div>
  `
})
export class ElectronicsComponent { }
```

```
// clothing.component.ts (Standalone)
import { Component } from '@angular/core';

@Component({
  selector: 'app-clothing',
  standalone: true,
  template: `
    <div>
      <h3>Clothing</h3>
```

```
      <ul>
        <li>Shirts</li>
        <li>Pants</li>
        <li>Shoes</li>
      </ul>
    </div>
  `
})
export class ClothingComponent { }
```

```
// books.component.ts (Standalone)
import { Component } from '@angular/core';

@Component({
  selector: 'app-books',
  standalone: true,
  template: `
    <div>
      <h3>Books</h3>
      <ul>
        <li>Fiction</li>
        <li>Non-Fiction</li>
        <li>Science</li>
      </ul>
    </div>
  `
})
export class BooksComponent { }
```

### Configuring Child Routes (Standalone)

```
// app.routes.ts (Standalone)
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { ProductsComponent } from './products.component';
import { ElectronicsComponent } from './electronics.component';
import { ClothingComponent } from './clothing.component';
import { BooksComponent } from './books.component';
import { PageNotFoundComponent } from './page-not-found.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
```

```
    path: 'products',
    component: ProductsComponent,
    children: [
      { path: 'electronics', component: ElectronicsComponent },
      { path: 'clothing', component: ClothingComponent },
      { path: 'books', component: BooksComponent }
    ]
  },
  { path: '**', component: PageNotFoundComponent }
];
```

**Output:**

**Browser Display (when navigating to /products/electronics):**

# Products Section

Electronics | Clothing | Books

---

## Electronics

- Laptops
- Smartphones
- Tablets

**Understanding Child Routes:**

- Parent component (ProductsComponent) has its own `<router-outlet>`
- Child components render inside the parent's router outlet
- URLs follow the pattern: `/parent/child` (e.g., /products/electronics)
- Parent component remains visible while child components change

## Router Configuration Methods

The way you configure routing differs between module-based and standalone approaches.

### Module-Based: RouterModule.forRoot() vs forChild()

In module-based apps, Angular provides two methods for configuring the RouterModule, each serving a different purpose.

#### RouterModule.forRoot()

- Used in the root module (AppModule)
- Registers router services globally
- Configures the router with application-wide routes
- Should be called only once in your entire application

```
// app-routing.module.ts (Root routing module)
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

#### RouterModule.forChild()

- Used in feature modules
- Does not register router services (they're already registered)
- Configures routes for a specific feature module
- Can be called multiple times in different feature modules

```
// products-routing.module.ts (Feature routing module)
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { ProductsComponent } from './products.component';
import { ProductListComponent } from './product-list.component';
import { ProductDetailComponent } from './product-detail.component';

const routes: Routes = [
```

```
  {
    path: 'products',
    component: ProductsComponent,
    children: [
      { path: '', component: ProductListComponent },
      { path: 'detail', component: ProductDetailComponent }
    ]
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class ProductsRoutingModule { }
```

```
// products.module.ts (Feature module)
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ProductsRoutingModule } from './products-routing.module';
import { ProductsComponent } from './products.component';
import { ProductListComponent } from './product-list.component';
import { ProductDetailComponent } from './product-detail.component';

@NgModule({
  declarations: [
    ProductsComponent,
    ProductListComponent,
    ProductDetailComponent
  ],
  imports: [
    CommonModule,
    ProductsRoutingModule  // Import feature routing module
  ]
})
export class ProductsModule { }
```

## Standalone: provideRouter() and Route Groups

With standalone components, there's no need for `forRoot()` or `forChild()`. Instead, you simply define route arrays and use `provideRouter()` in the bootstrap configuration (recommended for new projects).

### Step 1: Create Standalone Components

```
// home.component.ts (Standalone)
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div style="padding: 20px;">
      <h2>Welcome to TechCorp Solutions</h2>
      <p>We provide innovative software solutions for businesses worldwide.</p>
      <button routerLink="/services">View Our Services</button>
    </div>
  `
})
export class HomeComponent { }
```

```
// services.component.ts (Standalone)
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-services',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <div style="padding: 20px;">
      <h2>Our Services</h2>
      <nav>
        <a routerLink="web-development" routerLinkActive="active">Web Development</a> |
        <a routerLink="mobile-apps" routerLinkActive="active">Mobile Apps</a> |
        <a routerLink="consulting" routerLinkActive="active">Consulting</a>
      </nav>
      <hr>
      <router-outlet></router-outlet>
    </div>
  `
})
export class ServicesComponent { }
```

### Step 2: Define Routes

```
// app.routes.ts
```

### Step 2: Define Routes

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { ServicesComponent } from './services.component';
import { WebDevelopmentComponent } from './web-development.component';
import { MobileAppsComponent } from './mobile-apps.component';
import { ConsultingComponent } from './consulting.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'services',
    component: ServicesComponent,
    children: [
      { path: 'web-development', component: WebDevelopmentComponent },
      { path: 'mobile-apps', component: MobileAppsComponent },
      { path: 'consulting', component: ConsultingComponent }
    ]
  }
];
```

### Step 3: Bootstrap Configuration (Standalone)

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes)  // Single provider for all routes
  ]
});
```

⚠ Important:

**Standalone Routing Benefits:**

- No need for separate routing modules
- Simpler provider configuration
- Routes are just TypeScript exports
- Easier to understand and maintain
- Better tree-shaking for smaller bundles

## Practical Example: Building a Company Website

Let's create a complete routing setup for a company website with multiple pages and nested routes.

### Step 1: Create Components

```
// home.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  template: `
    <div style="padding: 20px;">
      <h2>Welcome to TechCorp Solutions</h2>
      <p>We provide innovative software solutions for businesses worldwide.</p>
      <button routerLink="/services">View Our Services</button>
    </div>
  `
})
export class HomeComponent { }
```

```
// services.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-services',
  template: `
    <div style="padding: 20px;">
```

```
      <h2>Our Services</h2>

      <nav style="margin: 20px 0;">
        <a routerLink="web-development" routerLinkActive="active-link">
          Web Development
        </a> |
        <a routerLink="mobile-apps" routerLinkActive="active-link">
          Mobile Apps
        </a> |
        <a routerLink="cloud-services" routerLinkActive="active-link">
          Cloud Services
        </a>
      </nav>

      <div style="border: 1px solid #ddd; padding: 15px; background: #f9f9f9;">
        <router-outlet></router-outlet>
      </div>
    </div>
  `,
  styles: [`
    .active-link {
      font-weight: bold;
      color: #007bff;
      text-decoration: underline;
    }
  `]
})
export class ServicesComponent { }
```

```
// web-development.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-web-development',
  template: `
    <div>
      <h3>Web Development Services</h3>
      <ul>
        <li>Custom Website Design</li>
        <li>E-commerce Solutions</li>
        <li>Content Management Systems</li>
        <li>Progressive Web Applications</li>
      </ul>
      <p>Starting from $5,000</p>
    </div>
```

```
  `
})
export class WebDevelopmentComponent { }
```

```
// mobile-apps.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-mobile-apps',
  template: `
    <div>
      <h3>Mobile Application Development</h3>
      <ul>
        <li>iOS App Development</li>
        <li>Android App Development</li>
        <li>Cross-Platform Solutions</li>
        <li>App Maintenance and Updates</li>
      </ul>
      <p>Starting from $10,000</p>
    </div>
  `
})
export class MobileAppsComponent { }
```

```
// cloud-services.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-cloud-services',
  template: `
    <div>
      <h3>Cloud Services</h3>
      <ul>
        <li>Cloud Migration</li>
        <li>Infrastructure as a Service</li>
        <li>Cloud Security</li>
        <li>24/7 Monitoring and Support</li>
      </ul>
      <p>Starting from $2,000/month</p>
    </div>
```

```
  `
})
export class CloudServicesComponent { }
```

```
// about.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-about',
  template: `
    <div style="padding: 20px;">
      <h2>About TechCorp Solutions</h2>
      <p>Founded in 2015, we have been delivering quality software solutions.</p>
      <p>Our team consists of 50+ experienced developers and designers.</p>
    </div>
  `
})
export class AboutComponent { }
```

```
// contact.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-contact',
  template: `
    <div style="padding: 20px;">
      <h2>Contact Us</h2>
      <p>Email: info@techcorp.com</p>
      <p>Phone: (555) 123-4567</p>
      <p>Address: 123 Tech Street, Silicon Valley, CA</p>
    </div>
  `
})
export class ContactComponent { }
```

## Step 2: Configure Routes

```
// app-routing.module.ts
import { HomeComponent } from './home.component';
import { ServicesComponent } from './services.component';
import { WebDevelopmentComponent } from './web-development.component';
```

```
import { MobileAppsComponent } from './mobile-apps.component';
import { CloudServicesComponent } from './cloud-services.component';
import { AboutComponent } from './about.component';
import { ContactComponent } from './contact.component';
import { PageNotFoundComponent } from './page-not-found.component';

const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  {
    path: 'services',
    component: ServicesComponent,
    children: [
      { path: 'web-development', component: WebDevelopmentComponent },
      { path: 'mobile-apps', component: MobileAppsComponent },
      { path: 'cloud-services', component: CloudServicesComponent }
    ]
  },
  { path: 'about', component: AboutComponent },
  { path: 'contact', component: ContactComponent },
  { path: '**', component: PageNotFoundComponent }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

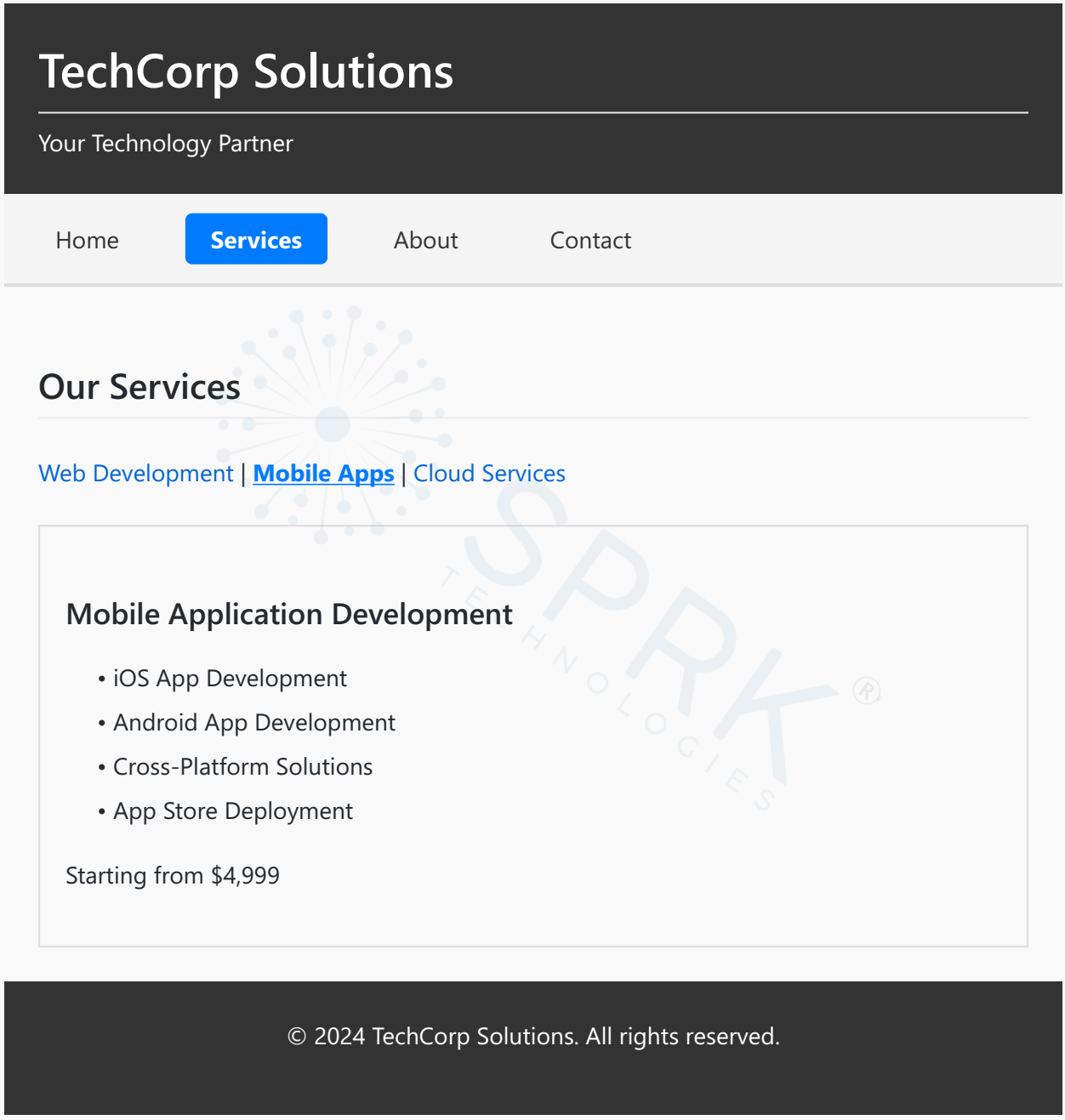
## Step 3: Create Main Application Component

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div style="font-family: Arial, sans-serif;">
      <header style="background: #333; color: white; padding: 20px;">
        <h1 style="margin: 0;">TechCorp Solutions</h1>
        <p style="margin: 5px 0 0 0;">Your Technology Partner</p>
      </header>

      <nav style="background: #f4f4f4; padding: 15px; border-bottom: 2px solid #ddd;">
```

```
        <a routerLink="/home"
           routerLinkActive="active"
           [routerLinkActiveOptions]="{exact: true}"
           style="margin-right: 20px; padding: 8px 15px; text-decoration: none; color: #333;">
          Home
        </a>
        <a routerLink="/services"
           routerLinkActive="active"
           style="margin-right: 20px; padding: 8px 15px; text-decoration: none; color: #333;">
          Services
        </a>
        <a routerLink="/about"
           routerLinkActive="active"
           style="margin-right: 20px; padding: 8px 15px; text-decoration: none; color: #333;">
          About
        </a>
        <a routerLink="/contact"
           routerLinkActive="active"
           style="padding: 8px 15px; text-decoration: none; color: #333;">
          Contact
        </a>
      </nav>

      <main style="min-height: 400px;">
        <router-outlet></router-outlet>
      </main>

      <footer style="background: #333; color: white; padding: 20px; text-align: center; margin-
top: 50px;">
        <p>© 2024 TechCorp Solutions. All rights reserved.</p>
      </footer>
    </div>
  `,
  styles: [`
    a.active {
      background-color: #007bff !important;
      color: white !important;
      border-radius: 4px;
      font-weight: bold;
    }
  `]
})
export class AppComponent { }
```

Output:

**Browser Display (when navigating to /services/mobile-apps):**



# Practical Task: Build a Blog Application

Create a blog application with the following requirements:

## Requirements

1. **Create the following components:**
   - `BlogHomeComponent` - Display welcome message and recent posts list
   - `AllPostsComponent` - Display list of all blog posts
   - `CategoriesComponent` - Parent component for categories
   - `TechnologyComponent` - Display technology posts
   - `LifestyleComponent` - Display lifestyle posts
   - `TravelComponent` - Display travel posts
   - `AuthorComponent` - Display author information
   - `PageNotFoundComponent` - 404 error page

2. **Configure routing with these paths:**
   - `/` - Redirect to `/home`
   - `/home` - Display BlogHomeComponent
   - `/posts` - Display AllPostsComponent
   - `/categories` - Display CategoriesComponent with child routes:
     - `/categories/technology` - Display TechnologyComponent
     - `/categories/lifestyle` - Display LifestyleComponent
     - `/categories/travel` - Display TravelComponent
   - `/author` - Display AuthorComponent
   - `/**` - Display PageNotFoundComponent (wildcard route)

3. **Navigation Requirements:**
   - Create a navigation bar in AppComponent with links to all main sections
   - Use `routerLink` directive for all navigation
   - Apply `routerLinkActive` to highlight the current page
   - Add sub-navigation in CategoriesComponent for category selection

4. **Styling Requirements:**
    - Style active navigation links with a distinct color
    - Create a consistent layout with header, navigation, content area, and footer
    - Make sure the router outlet is properly positioned

5. **Additional Features:**
    - Default the categories section to show technology posts when first visited
    - Add sample blog post titles and descriptions in each component
    - Include a "Back to Home" link in the 404 component

## Hints

- Start by creating all components using `ng generate component` command
- Define your routes array carefully, placing child routes inside the parent route
- Remember to use `pathMatch: 'full'` with redirect routes
- The wildcard route must be the last route in your array
- CategoriesComponent needs its own `<router-outlet>` for child routes
- Use array syntax for dynamic route parameters if needed
- Test each route by typing the URL directly in the browser address bar

## Expected Application Structure

```
src/
├── app/
│   ├── blog-home/
│   │   └── blog-home.component.ts
│   ├── all-posts/
│   │   └── all-posts.component.ts
│   ├── categories/
│   │   ├── categories.component.ts
│   │   ├── technology/
│   │   │   └── technology.component.ts
│   │   ├── lifestyle/
│   │   │   └── lifestyle.component.ts
│   │   └── travel/
│   │       └── travel.component.ts
│   ├── author/
│   │   └── author.component.ts
```

```
│   ├── page-not-found/
│   │   └── page-not-found.component.ts
│   ├── app-routing.module.ts
│   ├── app.component.ts
│   └── app.module.ts
```

### Testing Checklist

- ✓ Navigate to each route and verify the correct component loads
- ✓ Test all navigation links work correctly
- ✓ Verify active link highlighting is working
- ✓ Test child route navigation in the Categories section
- ✓ Navigate to a non-existent URL and verify 404 page appears
- ✓ Test the redirect from root path to /blog-home

## Key Concepts Summary

- **Module-Based:** Use `RouterModule.forRoot()` and `forChild()` for configuration
- **Standalone:** Use `provideRouter()` and simple route exports
- **<router-outlet>** is a placeholder where routed components are displayed
- **routerLink** directive enables navigation without page reloads
- **routerLinkActive** applies CSS classes to active route links
- **Child routes** create nested views with their own router outlets
- **Wildcard routes** handle invalid URLs (404 pages)
- **Redirects** automatically navigate from one path to another
- **Standalone components** simplify routing with less boilerplate

## Key Takeaways

1. Always use `routerLink` instead of `href` for internal navigation to maintain SPA behavior
2. Place wildcard routes last in the routes array to avoid matching all URLs
3. Use `exact: true` with routerLinkActiveOptions for root path links
4. Child routes require a router outlet in the parent component

5. Use `pathMatch: 'full'` with redirect routes to avoid partial matches
6. Organize routes logically to create intuitive URL structures
7. **For new projects:** Prefer standalone components for simpler code and better tree-shaking
8. **In standalone components:** Always import routing directives (RouterOutlet, RouterLink, etc.) in the imports array
9. **Module-based apps:** Still valid and widely used; understand both approaches for working with existing and new codebases